

Below is a FastAPI implementation using PostgreSQL-style raw SQL. It assumes the `users` and `tasks` tables from Step 3, with `tasks.user_id` referencing `users.id`.

```
from datetime import date, datetime
from typing import Optional

from fastapi import FastAPI, HTTPException, Path
from pydantic import BaseModel, Field
import psycopg2
from psycopg2.extras import RealDictCursor

app = FastAPI(title="Task Tracker API", version="1.0.0")

# -----
# Database
# -----

def get_db_connection():
    """Create a database connection."""
    return psycopg2.connect(
        host="localhost",
        database="task_tracker",
        user="postgres",
        password="your_password",
    )

# -----
# Request models
# -----

class TaskCreate(BaseModel):
    title: str = Field(..., min_length=1)
    description: Optional[str] = None
    due_date: Optional[date] = None

# -----
# Repository layer
# Keeps raw SQL separate from HTTP/controller logic (SRP).
# -----

class TaskRepository:

    @staticmethod
```

```

def create_task(
    user_id: int,
    title: str,
    description: Optional[str],
    due_date: Optional[date],
):
    sql = """
        INSERT INTO tasks (
            user_id,
            title,
            description,
            status,
            due_date,
            created_at
        )
        VALUES (%s, %s, %s, %s, %s, %s)
        RETURNING
            id,
            user_id,
            title,
            description,
            status,
            due_date,
            created_at;
    """

    with get_db_connection() as connection:
        with connection.cursor(cursor_factory=RealDictCursor) as cursor:
            cursor.execute(
                sql,
                (
                    user_id,
                    title,
                    description,
                    "pending",
                    due_date,
                    datetime.utcnow(),
                ),
            )
            return cursor.fetchone()

    @staticmethod
    def get_task_by_id(task_id: int):
        sql = """
            SELECT
                id,
                user_id,
                title,

```

```

        description,
        status,
        due_date,
        created_at
    FROM tasks
    WHERE id = %s;
"""

```

```

with get_db_connection() as connection:
    with connection.cursor(cursor_factory=RealDictCursor) as cursor:
        cursor.execute(sql, (task_id,))
        return cursor.fetchone()

```

```

# -----
# Controller / API routes
# -----

```

```

@app.post("/api/v1/tasks", status_code=201)
def create_task(task: TaskCreate):
    """

```

Create a new task.

In a real application, user_id should normally come from the authenticated user's JWT/session rather than the request body.

```

"""

```

```

# FastAPI/Pydantic parses the JSON request body into the TaskCreate
# object. For example:
# {
#   "title": "Finish API",
#   "description": "Implement task creation",
#   "due_date": "2026-09-30"
# }
#
# The date string is automatically parsed into a Python date object.
user_id = 1 # Replace with authenticated user's ID.

```

```

try:
    created_task = TaskRepository.create_task(
        user_id=user_id,
        title=task.title,
        description=task.description,
        due_date=task.due_date,
    )

```

```

return {
    "message": "Task created successfully",

```

```

        "task": created_task,
    }

except psycopg2.Error:
    # Do not expose database internals to the API client.
    raise HTTPException(
        status_code=500,
        detail="Unable to create task",
    )

@app.get("/api/v1/tasks/{id}")
def get_task(
    # FastAPI parses the URL path parameter /tasks/{id} into an integer.
    # A request such as GET /api/v1/tasks/42 therefore produces id=42.
    id: int = Path(..., gt=0)
):
    """Return a specific task by ID."""

    try:
        task = TaskRepository.get_task_by_id(id)

        if task is None:
            raise HTTPException(
                status_code=404,
                detail="Task not found",
            )

        return {
            "task": task,
        }

    except HTTPException:
        # Preserve intentional HTTP errors such as 404.
        raise

    except psycopg2.Error:
        # Hide database implementation details from API consumers.
        raise HTTPException(
            status_code=500,
            detail="Unable to retrieve task",
        )

```

Request examples

Create task

POST /api/v1/tasks

Content-Type: application/json

```
{
  "title": "Complete API implementation",
  "description": "Implement task creation and retrieval",
  "due_date": "2026-09-30"
}
```

Get task

GET /api/v1/tasks/42

Design notes

- **Raw SQL:** `INSERT` and `SELECT` are executed directly through `psycopg2`; no ORM is used.
- **SQL injection protection:** `%s` placeholders are used rather than interpolating request values into SQL.
- **DRY:** Database connection and repository responsibilities are centralized rather than duplicated in each route.
- **SOLID/SRP:** HTTP handling is separated from database access through `TaskRepository`.
- **Validation:** Pydantic validates `title` and parses `due_date`; `Path(gt=0)` validates the task ID.
- **Error handling:** Missing tasks return `404`; database failures return a generic `500` without exposing database details.
- **Authentication:** `user_id = 1` is intentionally a placeholder. In production, derive it from the authenticated request rather than accepting it from the client.